

Produtividade e Melhoria da DX (Developer eXperience)

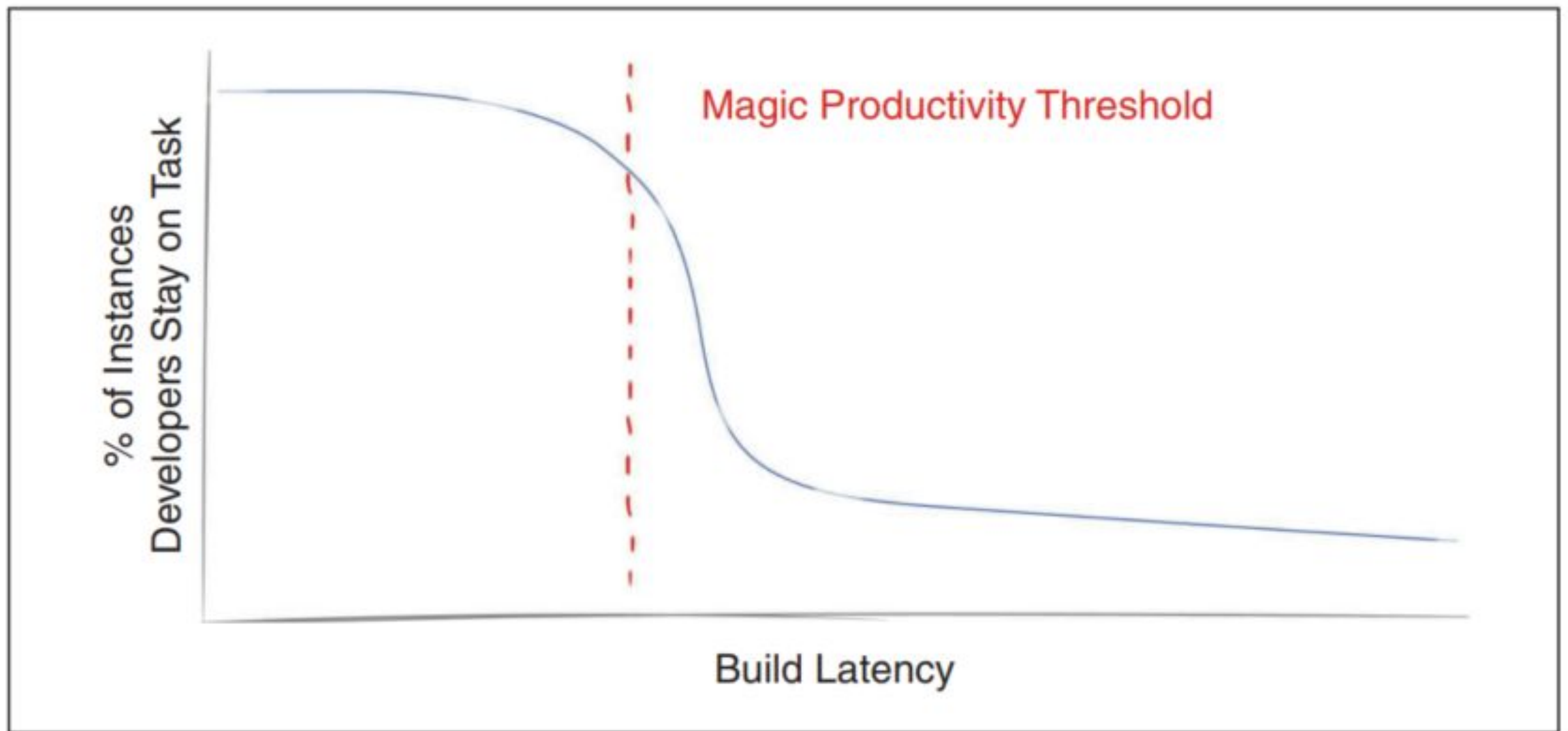
Patrícia G. F. Matsubara

Melhorando a experiência com o build

Latência de build

- Um **problema** de muitos!
 - Será que **reduzir** a latência traz benefícios?
- Ou **repensando** a questão:
 - Quão **rápidos** os builds precisam ser para o desenvolvedor permanecer na tarefa e se manter produtivo?

Suposição

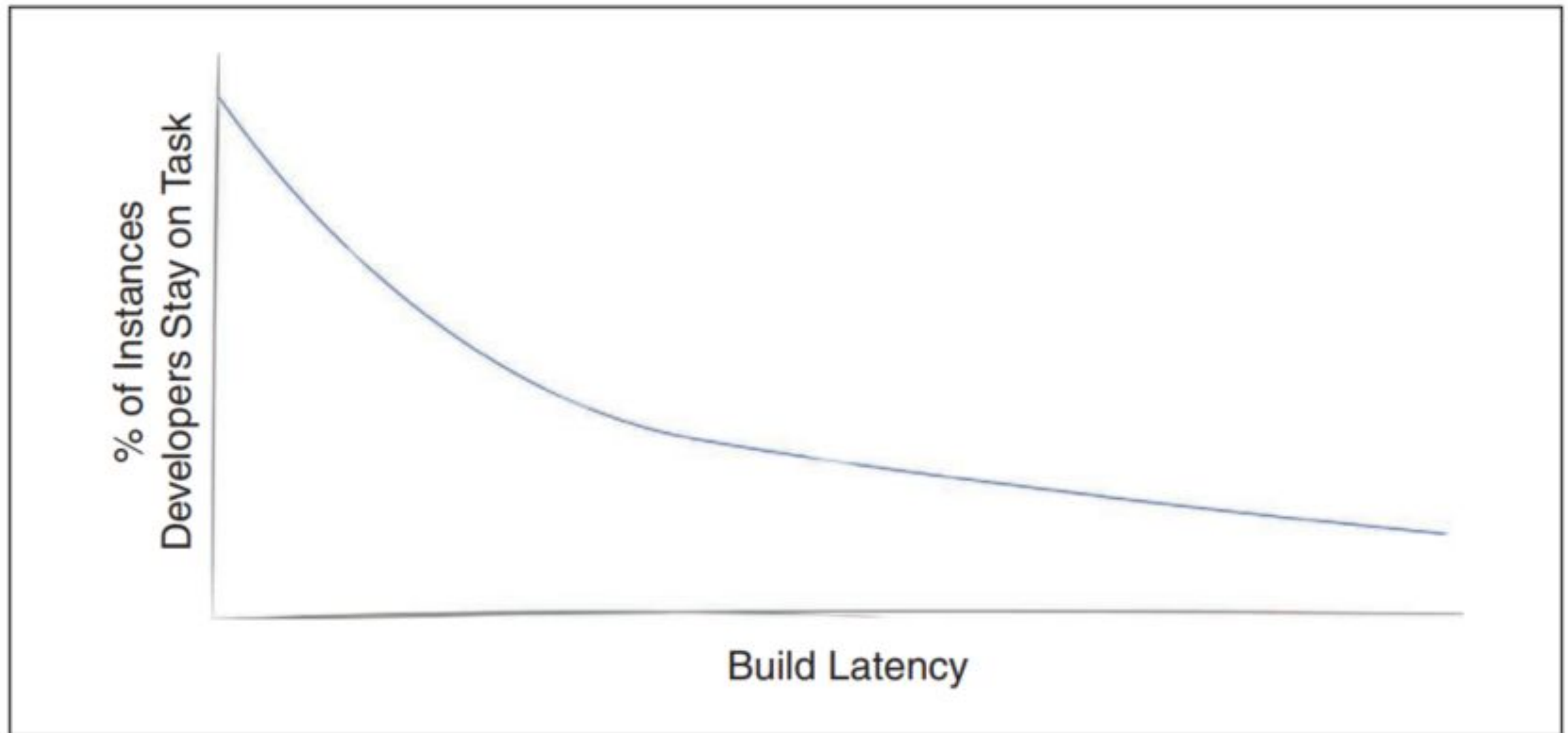


Hipótese da relação entre a latência do build e a probabilidade do desenvolvedor se manter na tarefa

Latência de build

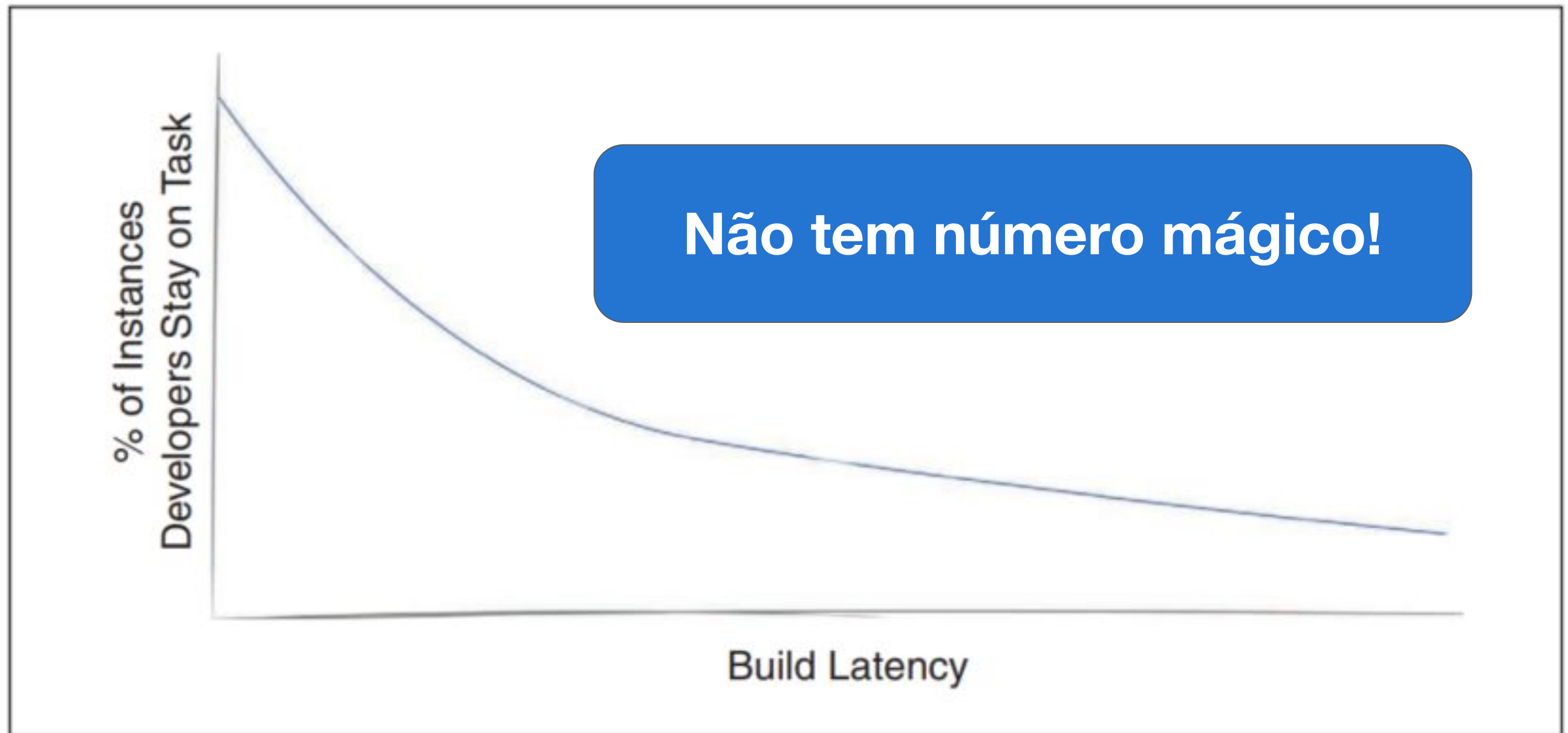
- **Relembrando...**
 - Quão **rápidos** os builds precisam ser para o desenvolvedor permanecer na tarefa e se manter produtivo?
- E **repensando** um pouco mais:
 - Durante um build quanto **tempo** leva para o desenvolvedor decidir **mudar** de tarefa (ir ver um email, trabalhar em outro projeto ou só parar a tarefa atual)?
 - Quando o build finaliza, quão **rápido** os desenvolvedores **voltam** para a tarefa e voltam a progredir nela?

Agora a realidade...



Hipótese da relação entre a latência do build e a probabilidade do desenvolvedor se manter na tarefa

Agora a realidade...



Hipótese da relação entre a latência do build e a probabilidade do desenvolvedor se manter na tarefa

Porquê não tem número mágico?

- **Porquê** o desenvolvedor...
 - Não **fica** na tarefa em builds curtos,
 - E **muda** de tarefa quando o build demora?
- Porque ele não sabe **quanto tempo** o build vai demorar!
 - Dificuldade de **estimativa!**

Estudo 1: estudo de amostragem

- Cada vez que um desenvolvedor **iniciava** um build
 - Mensagem de chat com um **survey** simples

Quanto tempo eles **esperavam** que a build levasse

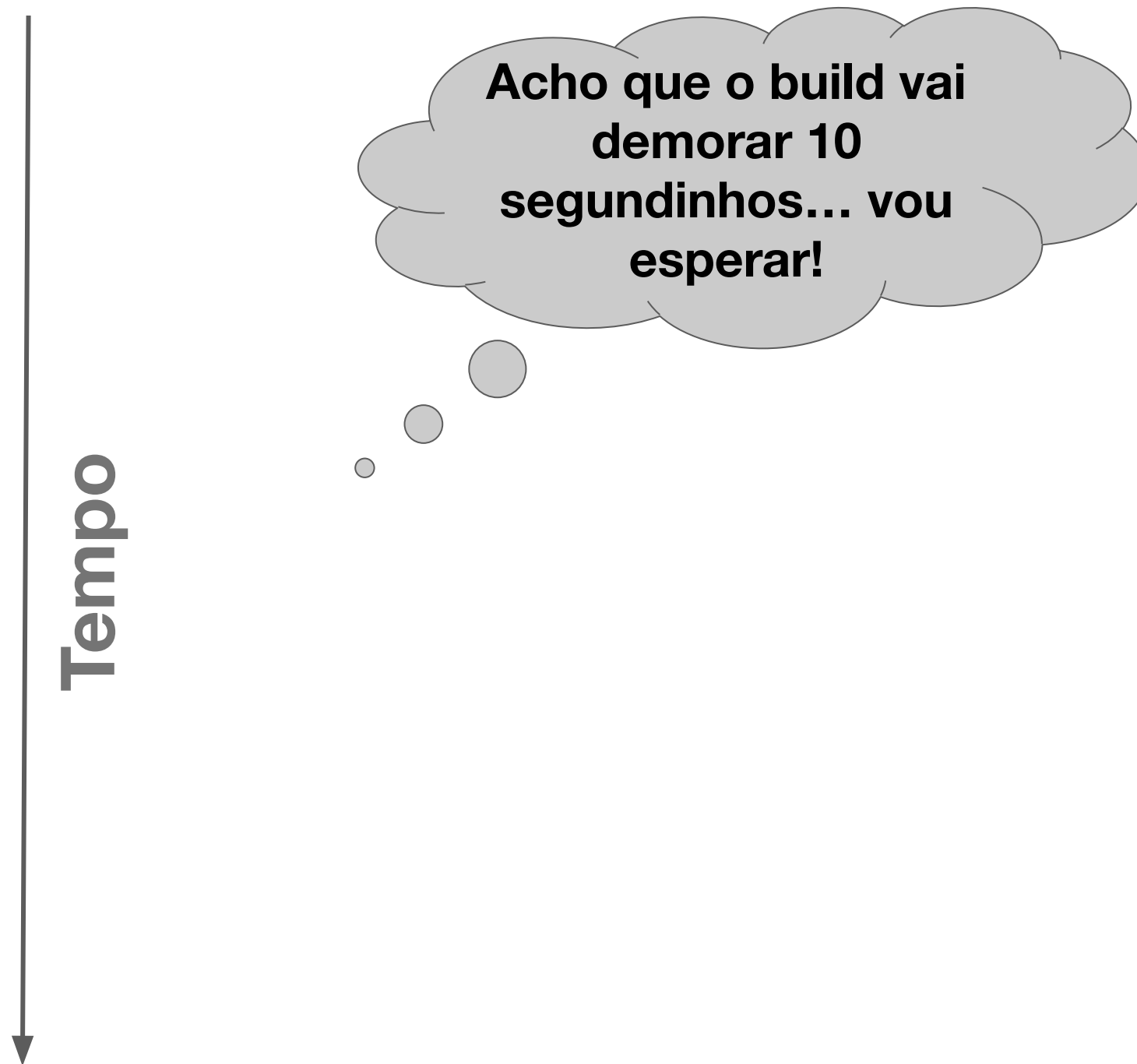
- “menos de 10 segundos”
- “menos de 1 minuto”,
- “1–2 minutos”,
- “2–5 minutos”
- “5–10 minutos”
- “10–20 minutos”
- “20–30 minutos”
- “30–60 minutos”
- “mais de 60 minutos”

O que eles estavam fazendo desde o **início** da build:

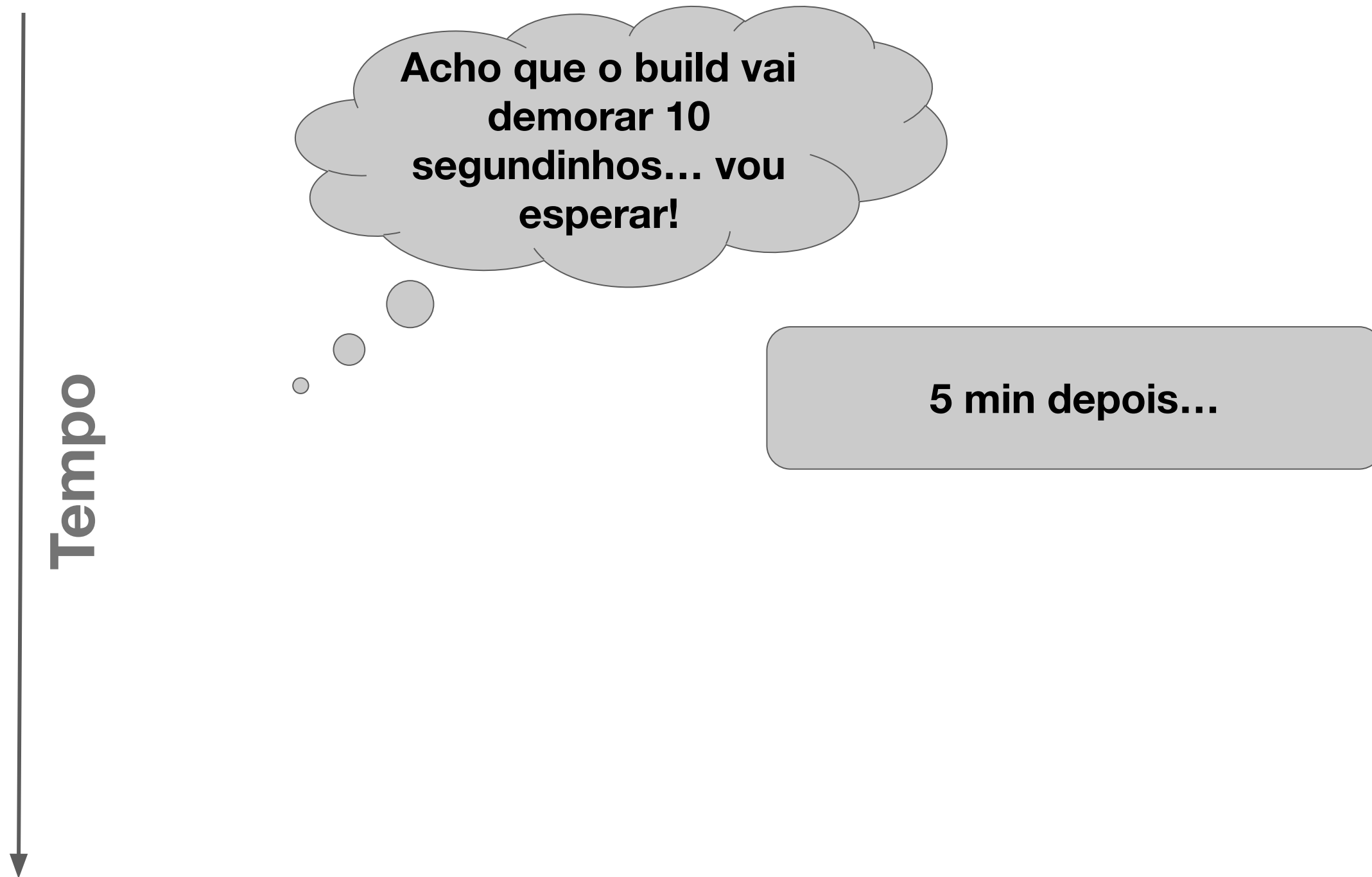
- “trabalhando nesta tarefa”,
- “trabalhando em outra tarefa”,
- “verificando e-mails”
- “atividade não relacionada ao trabalho”
- opção para resposta aberta

- Desenvolvedor escolhe o que fazer com base em suas **estimativas** sobre o tempo de build
- Problema: são muito **ruins** de estimar builds individuais!
 - Escolhem o bucket de tempo **errado** em **65%** dos casos
 - E se erram, ficam **frustrados** e sentem que a produtividade foi prejudicada!
- O dano vem mais da **imprevisibilidade** do tempo de build do que do seu tamanho

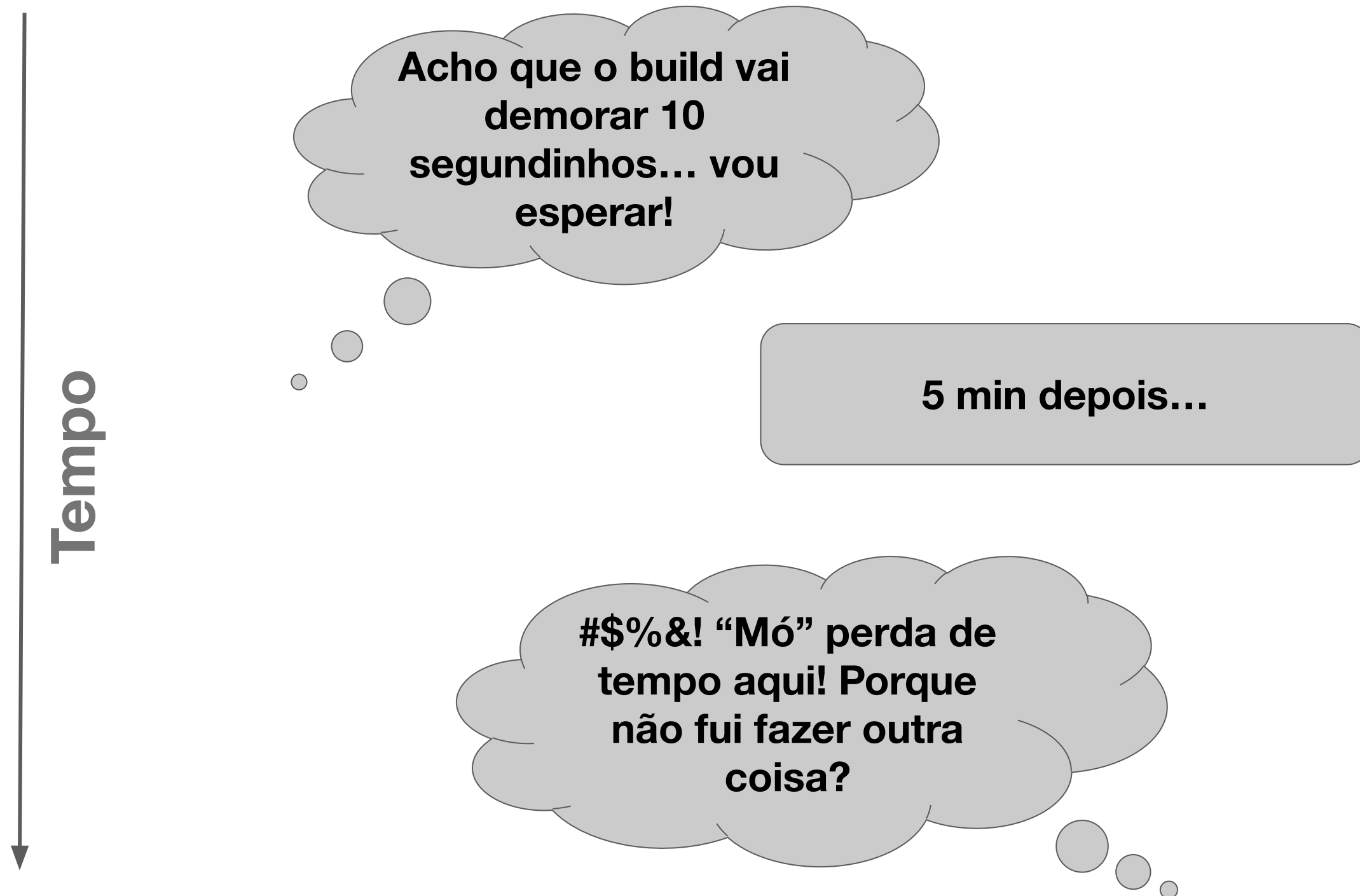
“Visualizando” - subestimativa



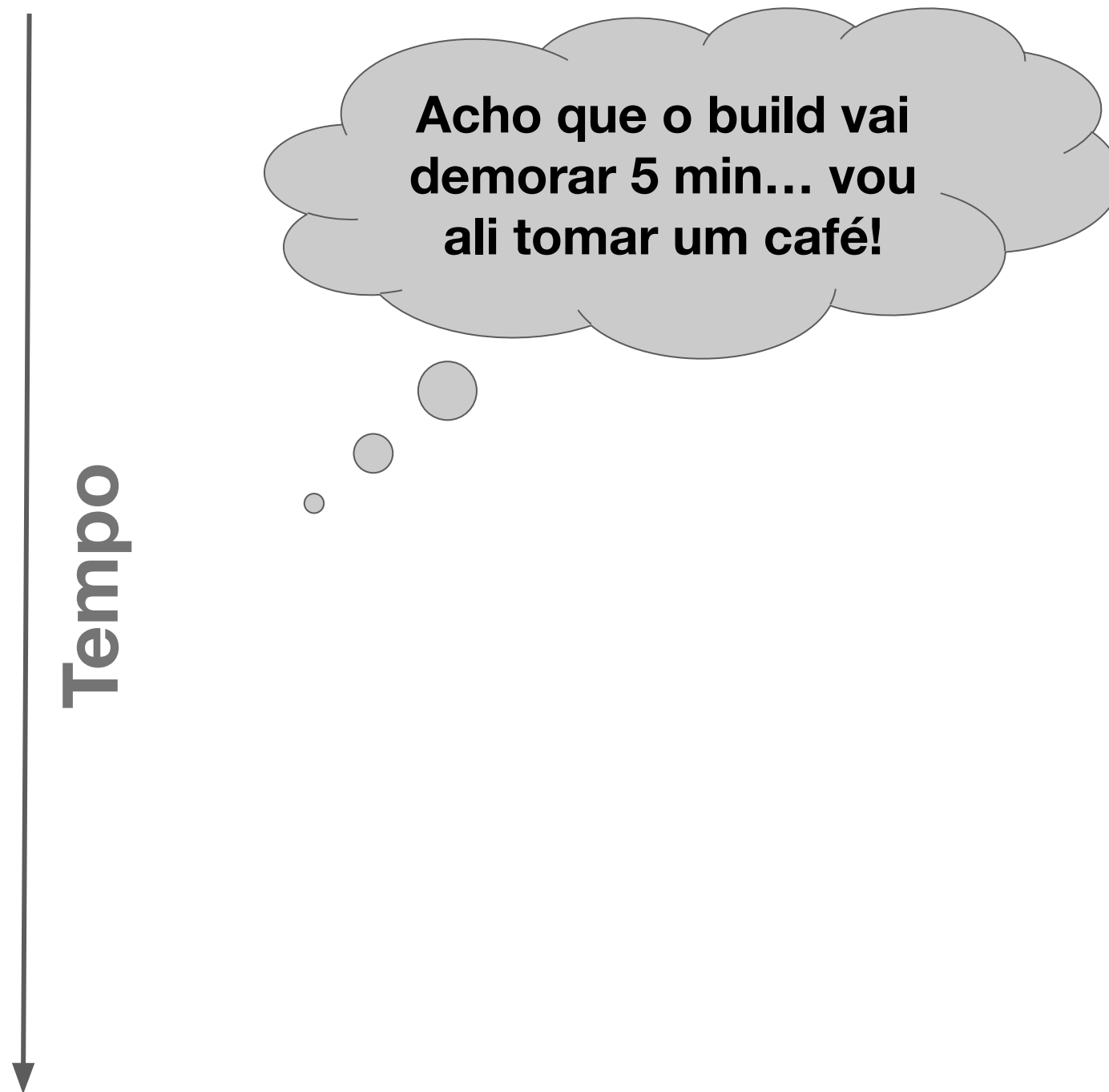
“Visualizando” - subestimativa



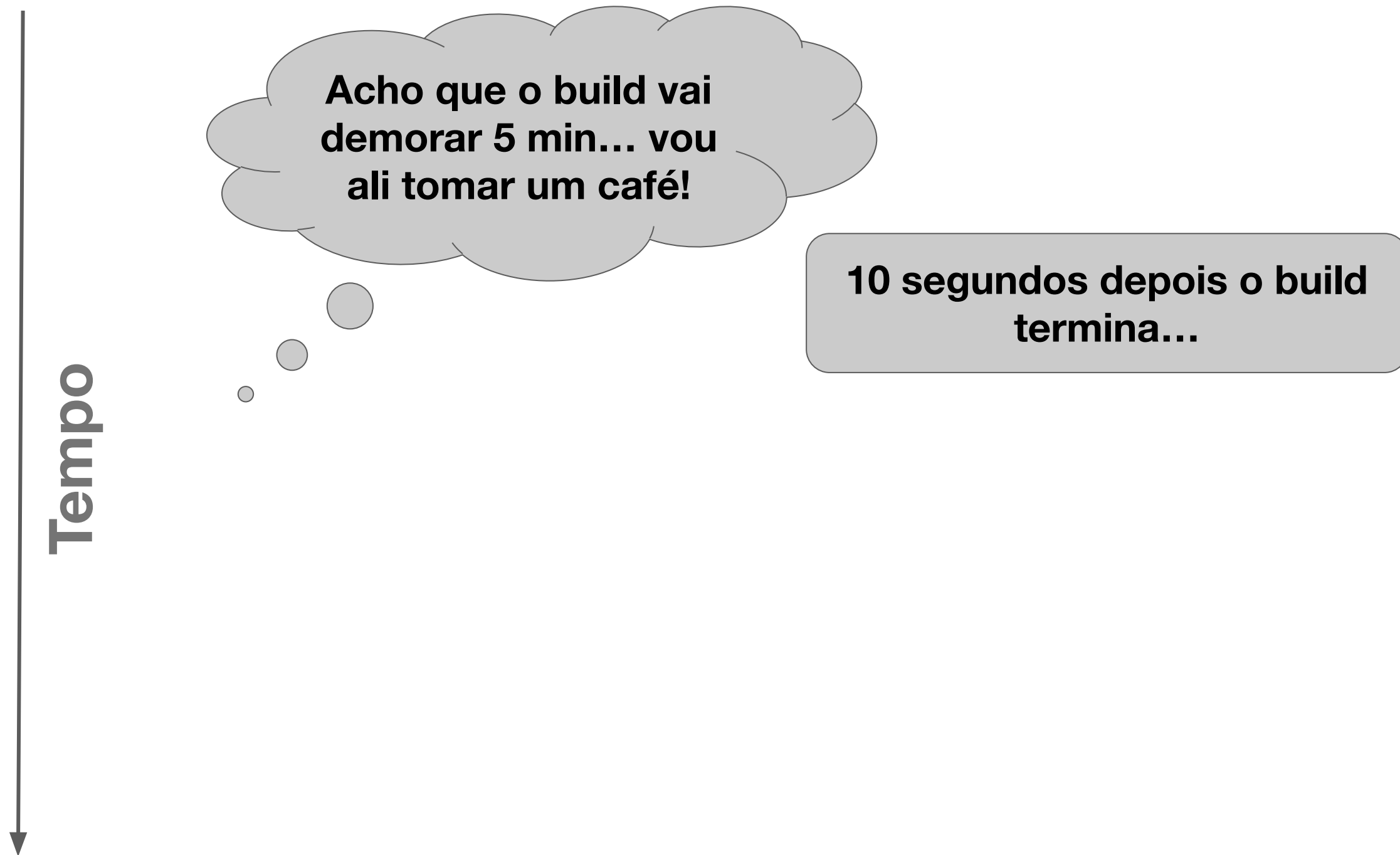
“Visualizando” - subestimativa



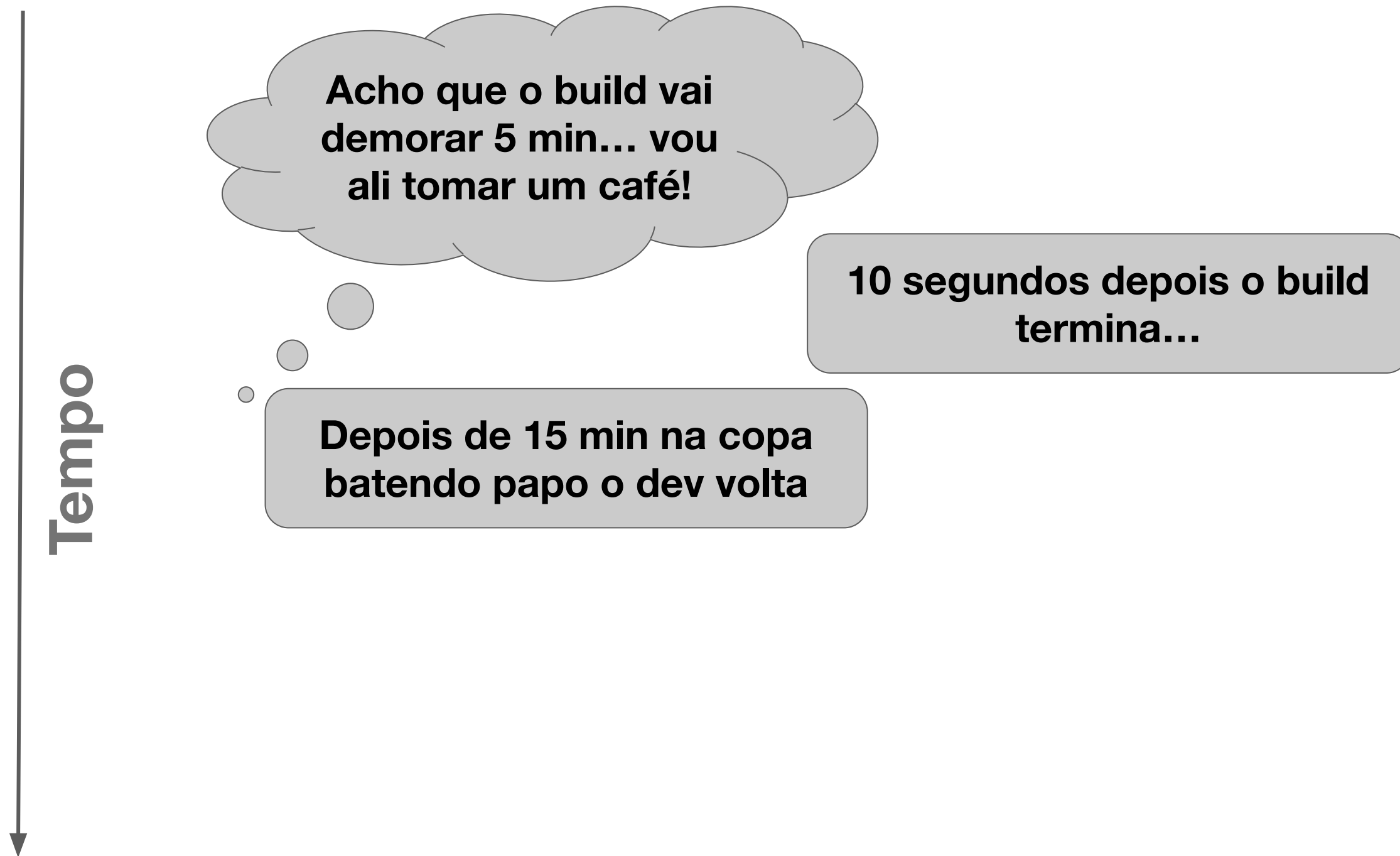
“Visualizando” - superestimativa



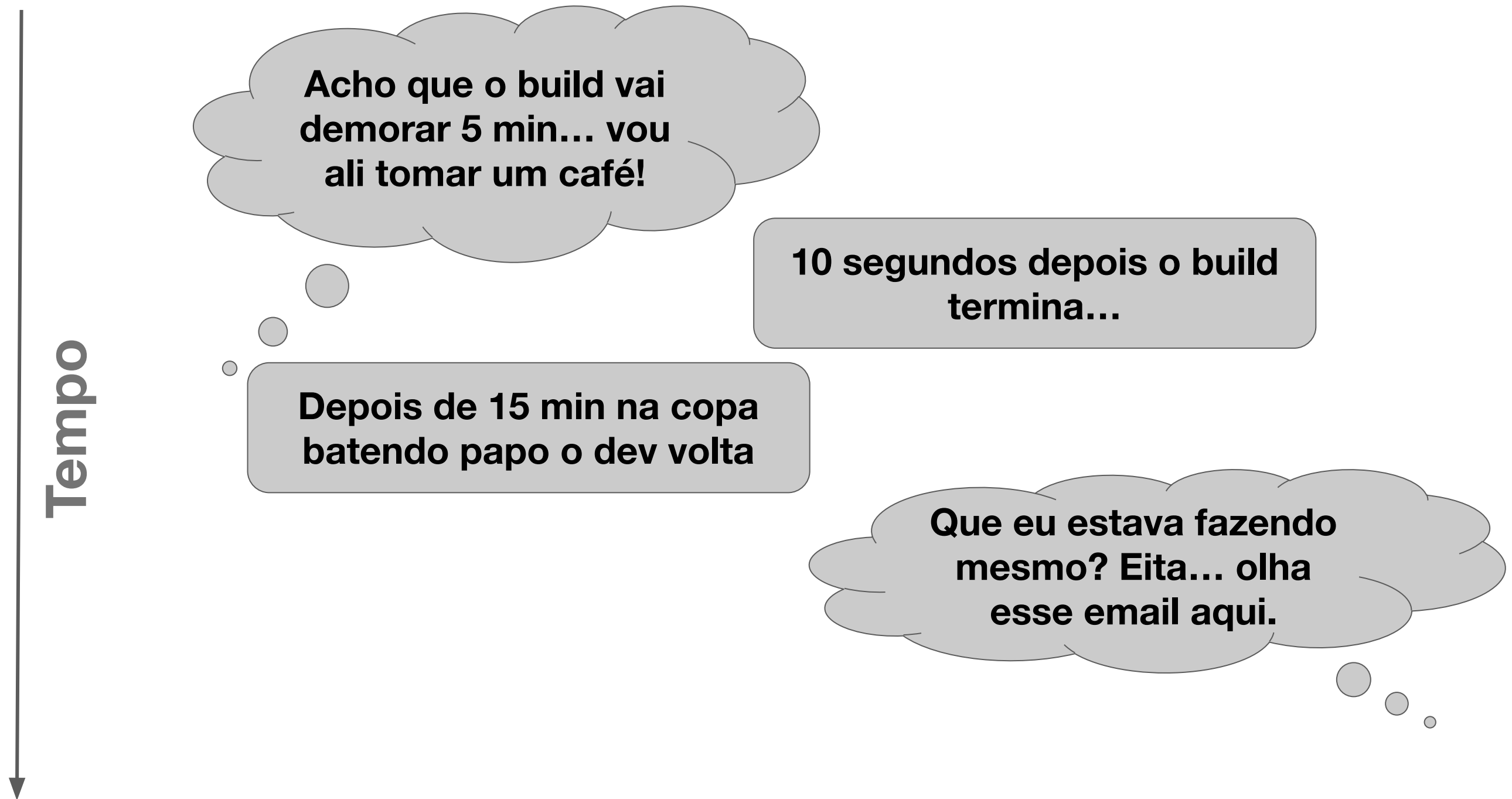
“Visualizando” - superestimativa



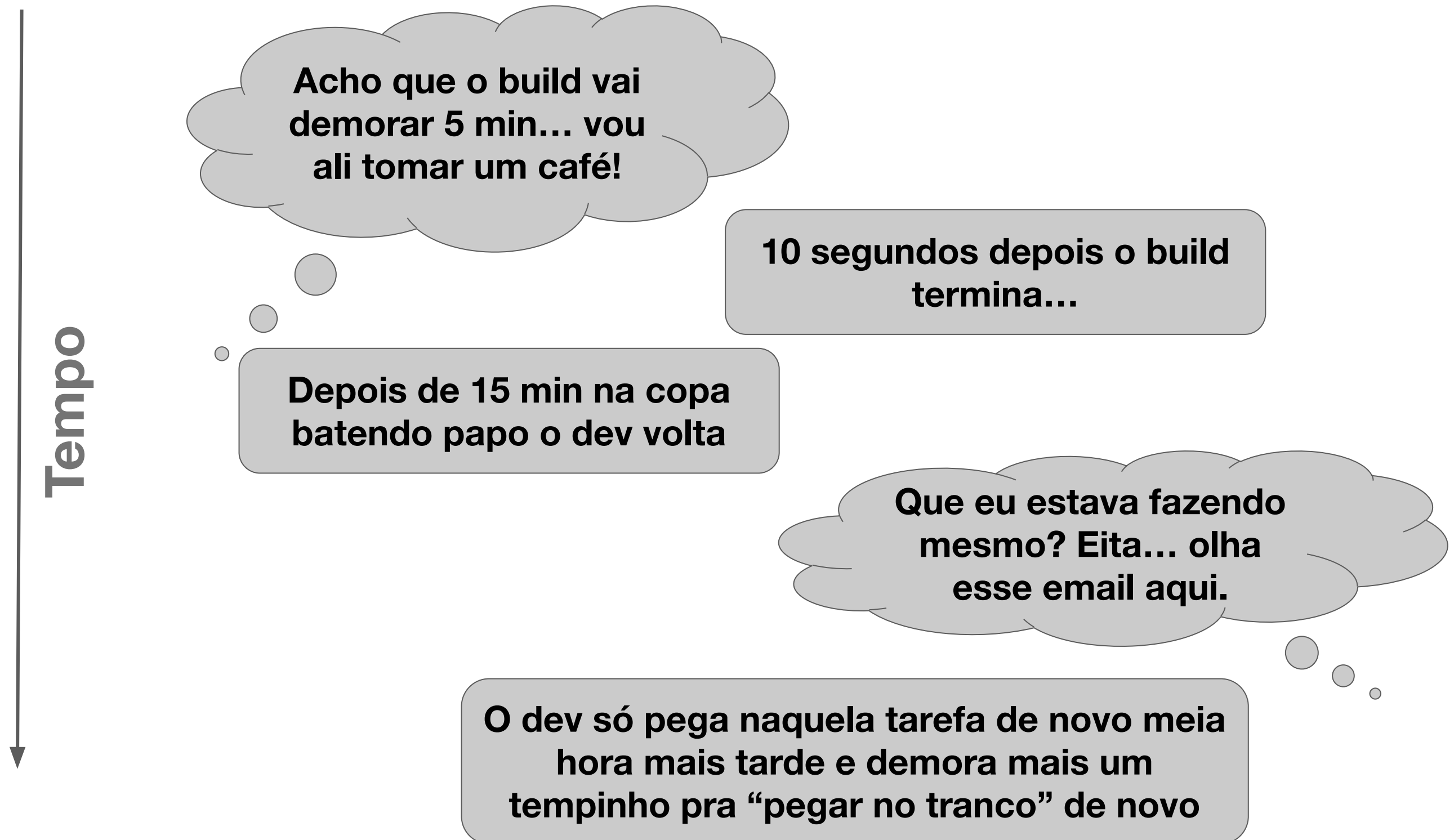
“Visualizando” - superestimativa



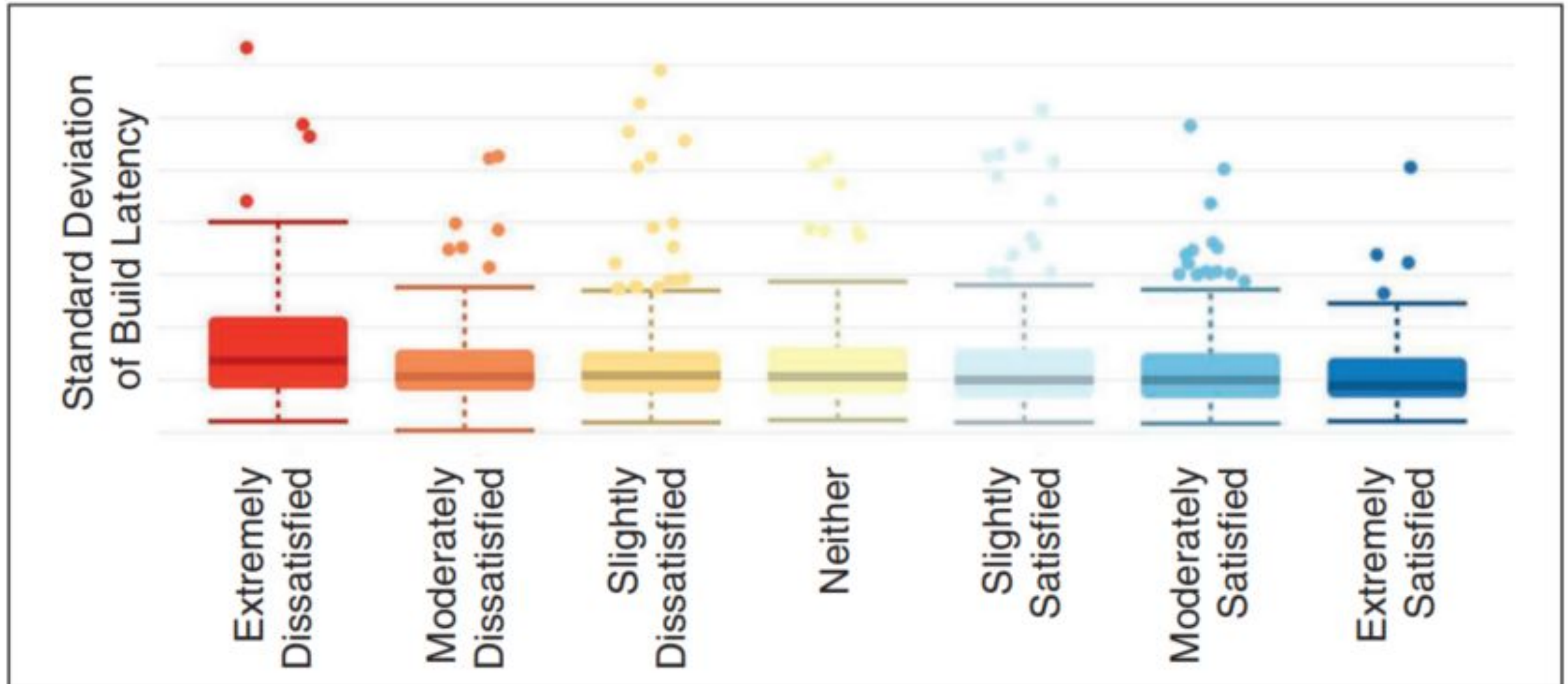
“Visualizando” - superestimativa



“Visualizando” - superestimativa



(Im)previsibilidade do build e satisfação



Desvio padrão da latência de build para cada engenheiro, particionado por nível de satisfação com a latência do build

Conclusões

- Tornar o tempo de build mais **previsível** seria o ideal
 - Ex.: tornar o tamanhos dos builds **similar**
 - Ex.: dar uma estimativa **confiável** do tempo de build para o desenvolvedor
- E também: mesmo melhorias **modestas** na latência de build são úteis
 - Vamos ver um **estudo** sobre isso...

Estudo 2: experimento controlado

- Estimar os **custos** de melhorar a infra para tornar o build mais rápido é fácil
- Difícil é estimar os benefícios!
- Equipe responsável pela infra de build **procurou** a equipe de pesquisa da Google
 - Já tinham adquirido máquinas que melhoravam o tempo em apenas **15%**
 - Parece pouco! Queriam saber se **valia a pena**

Estudo 2: experimento controlado

- **Questões** de pesquisa:
 - O desenvolvedor vai **notar** a maior rapidez?
 - Se notar, vai **mudar** seu comportamento?
- **Setup** do experimento
 - 15% dos desenvolvedores selecionados para o **grupo experimental** (sem saber)
 - 85% continuavam com as máquinas antigas - **grupo de controle**
- **Grupo experimental**
 - Melhoria de apenas **13%** na mediana do tempo de build
 - Ex.: Build de 1 min passava a durar ~ 51 seg

Coleta de dados

- **Survey** trimestral
 - **Produtividade** auto-relatada
 - **Velocidade** auto-relatada
 - **Satisfação** com a latência do build
- Métricas **baseadas em log**
 - Número de vezes que o dev **rodou um build**
 - Número de **linhas de código** submetidas
 - **Tempo de relógio** gasto com codificação nas listas de mudanças submetidas
 - **Tempo ativo** (*“fingers on the keyboard time”*) de codificação nas listas de mudança submetidas

Resultados

- **Survey** trimestral
 - **Produtividade** auto-relatada: **+4%** do GE moderadamente produtivos
 - **Velocidade** auto-relatada: **+5%** do GE satisfeitos com sua velocidade
 - **Satisfação** com a latência do build: **+5%** do GE satisfeitos com a latência do build

Resultados

- **Survey** trimestral
 - **Produtividade** auto-relatada: **+4%** do GE moderadamente produtivos
 - **Velocidade** auto-relatada: **+5%** do GE satisfeitos com sua velocidade
 - **Satisfação** com a latência do build: **+5%** do GE satisfeitos com a latência do build

Pode parecer pouco, mas lembrem que as **melhorias no sistema de build foram modestas!**

Resultados

- Métricas **baseadas em log**
 - Número de vezes que o dev **rodou um build: um build a mais por semana** no GE
 - Número de **linhas de código** submetidas: **24 linhas de código a mais por semana** no GE
 - **Tempo de relógio** gasto com codificação nas listas de mudanças submetidas: GE **14% mais rápidos** em listas pequenas-médias
 - **Tempo ativo** (*“fingers on the keyboard time”*) de codificação nas listas de mudança submetidas: GE **11% mais rápidos** em listas pequenas-médias

Resultados

- Métricas **baseadas em log**
 - Número de vezes que o dev **rodou um build: um build a mais por semana** no GE
 - Número de **linhas de código** submetidas: **24 linhas de código a mais por semana** no GE
 - **Tempo de relógio** gasto com codificação nas listas de mudanças submetidas: GE **14% mais rápidos** em listas pequenas-médias
 - **Tempo ativo** (*"fingers on the keyboard time"*) de codificação nas listas de mudança submetidas: GE **11% mais rápidos** em listas pequenas-médias

Mas essas melhorias só aparecerem no **terceiro mês do experimento!**

Resultados

- Mudanças nas métricas **persistiram** ainda por mais dois meses
- Mostra que os desenvolvedores levaram um tempo para se **adaptar às melhorias**

Conclusões

- Mudança na latência do build requer melhorias **técnicas**
- Mas a questão **humana** importa muito!
 - O ser humano precisa **perceber** a mudança na latência
 - Integrar essa percepção às suas **expectativas**
 - E **alterar** seu dia de trabalho de acordo

Mais sobre o survey

Engineering Satisfaction Survey (**EngSat**)

DEVELOPER PRODUCTIVITY
FOR HUMANS



Editor: Ciera Jaspan
Google
ciera@google.com



Editor: Collin Green
Google
colling@google.com

Measuring Developer Experience With a Longitudinal Survey

Sarah D'Angelo^{ID}, Jessica Lin, Jill Dicker, Carolyn Egelman^{ID}, Maggie Hodges^{ID},
Collin Green^{ID}, and Ciera Jaspan^{ID}

Engineering Satisfaction Survey (EngSat)

- **Survey**
 - Abordagem de **pesquisa**
 - Baseada em **questionários**
 - Respondido por uma **amostra** da população
 - Para descrever alguma **característica** ou explicar algum fenômeno
- Realizado **trimestralmente** desde 2018 na Google
 - *“It has been one of our most robust data sources”*

Por que um survey? Vantagens

- Centrado no **ser humano**
 - Como **percebem, pensam e se sentem**
 - Em relação ao mundo
- **Flexíveis e rápidos**
 - Mais **fácil** criar uma questão em um survey
 - Do que criar uma métrica baseada em log
- Disponibilizam dados de experiências que não dá para medir **objetivamente**
 - Ex.: satisfação

Por que um survey? Vantagens

- Permitem questões **abertas**
 - Mais **insights**
- Podem rodar com equipes **pequenas e grandes**
- Requerem **pouca infraestrutura** para funcionar
 - E menos **instrumentação**
 - Por isso é um bom método de **entrada** para medir produtividade

A experiência da Google

- O EngSat foi lançado **para**
 - Compreender as **necessidades** dos engenheiros de software
 - **Complementar** as métricas baseadas em logs
- Buscaram se **diferenciar** de outros surveys para funcionários
 - Focaram em experiências **únicas** aos desenvolvedores
 - Perguntaram sobre **ferramentas** internas e **tarefas** específicas de desenvolvimento
 - Enfatizaram tópicos **difíceis de medir** (ex.: dívida técnica, fluxo, qualidade)
 - E que **não cobertos** pelas métricas baseadas em logs (satisfação ou obstáculos a produtividade) → para servir de bases para pesquisas futuras

A experiência da Google

- Se juntaram com **experts** para criar questões representativas
 - dos **fluxos** dos desenvolvedores

Nem tudo são flores...

- Enfrentaram **resistência e críticas**. Ex.: surveys são
 - **Subjetivos** (sujeitos a erros e interpretações)
 - **Enviesados** (respondentes podem ter incentivos para não dizer a verdade)
 - Mais fiéis ao que as pessoas **sentem** do que à realidade
- Principalmente quando há resultados **negativos** ou **contra intuitivos**
- **Antídotos** para desafiar essas percepções
 - **Parear** os resultados com as métricas baseadas em logs
 - Execução do survey **regular**, com cadência **previsível** e com substância **consistente**
 - Permite demonstrar o valor de dados **longitudinais** e reforçar a utilidade do programa

Características chave do EngSat

- **Staffing** consistente e adequado é **chave**
- Um pesquisador de UX e um engenheiro são **donos** do programa
 - **Pesquisador de UX:** direciona a evolução e gerenciamento do instrumento; direciona as operações e comunicação do programa
 - **Engenheiro:** direciona a infraestrutura de processamento e análise automatizadas de dados
- Esquema de **rotação**
 - Mudam a **cada trimestre**
 - Incentivo para ter um processos **repetíveis** e bem **documentados**
 - Ajuda no gerenciamento de **stakeholders**

Características chave do EngSat

- O próprio **processo** é **chave**

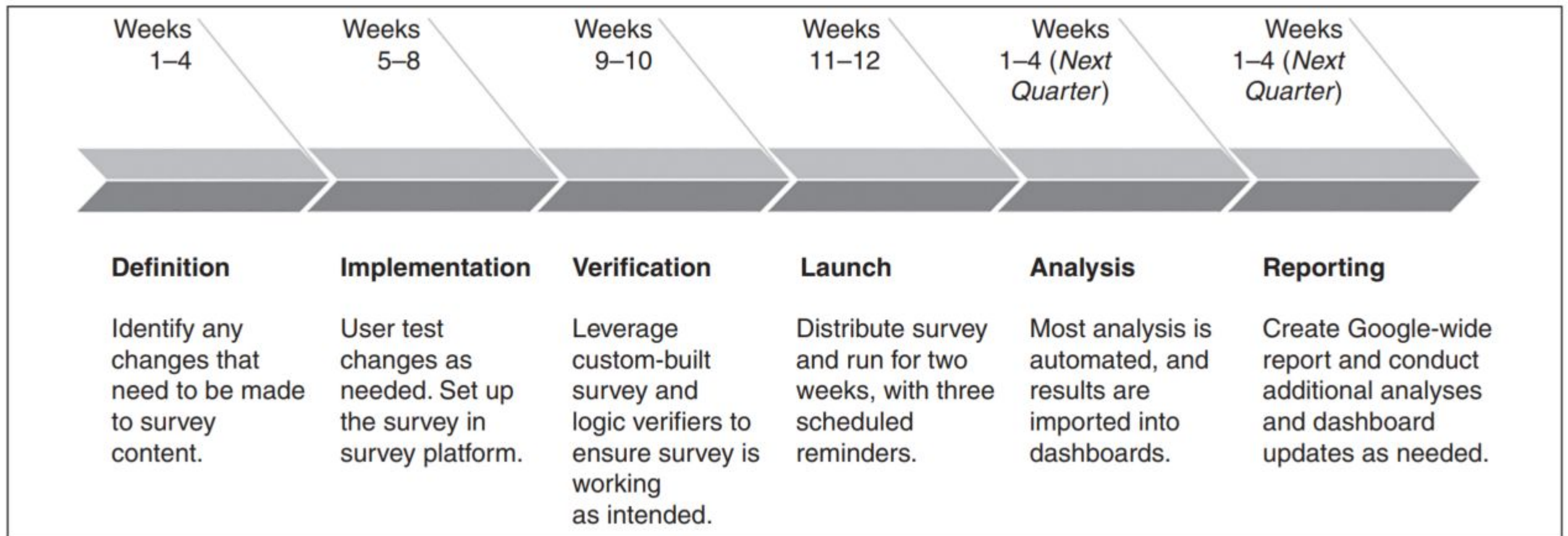


FIGURE 1. The end-to-end process for running the EngSat survey.

Características chave do EngSat

- Cada **fase** tem
 - **Documentação** de referência
 - **Exemplos** de execuções anteriores do survey
 - **Templates**
 - E, se apropriado, código e **automação** de análise

Características chave do EngSat

- A **automação** também é **chave**
- **Infraestrutura** para limpar e agregar dados
 - E disponibilizar em **dashboards e relatórios**

Mais sobre a infra

- Abordagem consistente para **formatação** dos dados
 - Formato de **armazenamento único** para tipos de questões comuns
 - Permite **combinar dados** de múltiplas execuções do survey e **adaptar** a ferramentas diferentes de surveys
- Uso de **scripts** padronizados
 - Script **verificador** para garantir que o survey lançado é compatível com a automação e checa mudanças em relação a execuções anteriores
 - Script para **puxar** os dados da ferramenta, converter para o formato de armazenamento e colocar nas tabelas
 - Scripts de **agregação e análise** que calculam automaticamente intervalos de confiança e analisam *nonresponse biases*

Exemplo de uso dos dados

Produtividade na pandemia

- Já tinham dados de **dois anos completos do EngSat** quando a pandemia começou
 - Serviu de **baseline de comparação** para antes, durante e depois da pandemia
- Resultado **imediato** sobre produtividade na pandemia
 - **Queda notável** na produtividade auto-reportada entre Q1 2020 e Q2 2020
 - Pois as pessoas começaram **WFH** (*work from home*)
- **Rapidamente** mudaram o EngSat
 - Inclusão de questões **abertas**
 - Como as pessoas estavam se **adaptando** e como a Google podia **ajudar**

Produtividade na pandemia

- **Insights** das questões abertas
 - Levaram a **melhorias** no acesso remoto e conectividade
- Com o tempo os níveis de produtividade **voltaram** aos pré-pandêmicos
 - E até o **excederam**
- Resumindo: ter o survey **em execução** ajudou a
 - **Compreender rapidamente** o impacto de um evento global
 - **Propor** mudanças para adaptar
 - E **avaliar** as melhorias

Obrigada!



That's all folks!

